



南开大学
Nankai University

南 开 大 学

数 学 科 学 学 院

机器学习期末实验报告

空气压力系统故障检测

谢 博

年级：2024 级

专业：数学与人工智能

指导教师：刘杰

2026 年 6 月 7 日

摘要

本实验围绕重型卡车空气压力系统 (APS) 故障检测任务, 构建了一个面向不平衡二分类场景的传统机器学习分类流程。数据集包含 60000 条训练样本、170 个匿名传感器特征和 1 个类别标签, 其中正类样本仅占约 1.67%。由于题目给出的维修代价函数将一次漏报 (FN) 的代价设为一次误报 (FP) 的 50 倍, 实验目标并非单纯追求准确率, 而是在保持可接受误报数量的同时尽量减少漏报。本文采用 scikit-learn 中的 HistGradientBoostingClassifier 处理表格型传感器特征和缺失值, 并在分层验证集上以 $10 \times FP + 500 \times FN$ 为目标搜索分类阈值。实验最终选取阈值 0.0110, 在验证集上取得 5320 的 APS Cost、0.9800 的 Recall 和 0.5385 的 F1。在同一份训练数据评测中, 模型相较 baseline 将 Cost 从 38140 降低到 16940, 说明代价敏感的阈值选择可以有效改善不平衡故障检测任务的实际目标。本次作业相关代码已上传至 github, 仓库地址为 <https://github.com/Zemiss/ImbBin-Coursework>

关键字: 空气压力系统; 不平衡二分类; 梯度提升树; 阈值搜索; 代价敏感学习

目录

一、 实验目的	1
(一) 实验背景	1
(二) 实验目标	1
二、 实验原理	1
(一) 数据处理	1
(二) 模型选择	1
(三) 阈值选择	2
三、 实验步骤	2
(一) 整体流程	2
(二) 配置与运行	2
四、 程序代码	3
(一) 数据读取与标签转换	3
(二) 模型构建与代价函数	3
(三) 阈值搜索与指标评估	4
(四) 训练与测试入口	4
五、 实验结果显示	5
(一) 验证集结果	5
(二) 与 baseline 对比	5
六、 实验分析总结	5

一、实验目的

(一) 实验背景

空气压力系统 (Air Pressure System, APS) 是重型卡车制动和辅助控制中的关键子系统。若 APS 相关故障未能及时识别, 车辆可能产生更高的维修成本和安全风险。本实验使用的数据来自 APS 故障检测二分类任务, 标签列为 `class`, 其中 `pos` 表示故障与空气压力系统相关, `neg` 表示故障与空气压力系统无关。

该任务的主要困难在于类别分布极不平衡。训练数据共有 60000 条样本, 其中 `neg` 为 59000 条, `pos` 为 1000 条, 正类比例约为 1.67%。在这种数据分布下, 如果模型只追求整体准确率, 可能倾向于预测多数类, 从而漏掉真正的 APS 相关故障。因此, 本实验将题目定义的维修代价作为更重要的优化目标。

(二) 实验目标

本实验的目标是完成一个可复现的 APS 故障检测流程, 包括数据读取、标签转换、模型训练、阈值选择、模型保存和测试评估。题目给出的维修代价函数为:

$$Cost = 10 \times FP + 500 \times FN \quad (1)$$

其中, `FP` 表示将无关故障误判为 APS 相关故障, `FN` 表示将 APS 相关故障漏判为无关故障。由于一次 `FN` 的代价相当于 50 次 `FP`, 本实验在模型评估时更加关注 `Recall` 和 `FN` 数量, 并通过阈值搜索直接降低总体 `Cost`。

二、实验原理

(一) 数据处理

实验数据存放于 `data/ml_train.csv`, 任务来源于 APS Failure at Scania Trucks 数据集 [2]。每条样本包含 170 个匿名传感器特征和 1 个标签列。数据中的缺失值以字符串 `na` 表示, 程序读取 CSV 时将其转换为 `NaN`。标签列 `class` 被转换为二进制数值标签, 即 `pos` 转换为 1, `neg` 转换为 0。

为了保证验证集能够反映原始类别比例, 训练阶段使用分层抽样划分训练子集和验证子集。默认配置中 `valid_size=0.2`, 因此 80% 的样本用于拟合验证模型, 20% 的样本用于选择分类阈值。

(二) 模型选择

本实验采用 `scikit-learn` [1] 中的 `HistGradientBoostingClassifier` 作为分类模型。该模型属于梯度提升树方法, 能够处理表格型数据中的非线性关系, 并且可以直接处理缺失值。相比需要额外归一化和缺失值填补的模型, 该方法更适合当前包含大量传感器字段和缺失值的任务。

模型主要参数如表 1 所示。随机种子固定为 42, 以保证数据划分和模型训练过程可复现。

参数	取值	说明
<code>max_iter</code>	500	最大 boosting 迭代轮数
<code>learning_rate</code>	0.04	每轮弱学习器的学习率
<code>max_leaf_nodes</code>	31	每棵树的最大叶子节点数
<code>l2_regularization</code>	0.03	L2 正则化强度
<code>random_state</code>	42	随机种子

表 1: HistGradientBoostingClassifier 主要参数

(三) 阈值选择

普通二分类模型通常使用 0.5 作为分类阈值，但该策略并不适合本实验的代价函数。因为 FN 的代价远高于 FP，模型需要更积极地识别 pos 类样本。实验在验证集上搜索候选阈值，将正类概率转换为预测标签，并选择 APS Cost 最低的阈值。

候选阈值由两段网格组成：第一段为 0.001 到 0.1，第二段为 0.11 到 0.9。最终选出的分类阈值为 0.0110。较低的阈值提升了模型对正类的敏感性，从而有效减少漏报。

三、实验步骤

(一) 整体流程

实验流程如算法 1 所示。训练阶段先在训练子集上拟合验证模型，并在验证集上搜索最佳阈值。确定阈值后，程序使用全量训练数据重新训练最终模型，并将模型、阈值、特征列顺序和验证指标保存到 `main/models/model.joblib`。

Algorithm 1 APS 故障检测训练与评估流程

Input: 训练数据 `data/ml_train.csv`，维修代价函数 $Cost = 10FP + 500FN$

Output: 最终模型、分类阈值和评估指标

- 1: 读取 CSV，并将 na 转换为缺失值
 - 2: 将 class 中的 pos/neg 转换为 1/0
 - 3: 使用分层抽样划分训练子集和验证集
 - 4: 在训练子集上拟合 HistGradientBoostingClassifier
 - 5: 在验证集上输出正类概率
 - 6: 遍历候选阈值，计算每个阈值对应的 APS Cost
 - 7: 选择 Cost 最低的阈值作为最终分类阈值
 - 8: 使用全量训练数据重新训练最终模型
 - 9: 保存模型包并在测试阶段加载评估
-

(二) 配置与运行

项目默认配置文件为 `main/configs/default.yaml`。训练数据路径、模型保存路径、验证集比例和随机种子均由配置文件给出，也可以通过命令行参数临时覆盖。

训练命令

```
1 python -m main.train
```

测试命令

```
1 python -m main.test
```

当测试数据包含 class 标签列时, 程序会输出 Cost、Precision、Recall、F1、Micro-F1、Macro-F1 和混淆矩阵。当测试数据不包含标签列且指定 prediction_path 时, 程序会输出每条样本的正类概率和预测类别。

四、 程序代码

(一) 数据读取与标签转换

main/src/modeling.py 封装了公共建模函数。数据读取函数将字符串 na 识别为缺失值, 标签转换函数将 pos 和 neg 转换为二分类数值标签。

数据读取与标签转换

```
1 POS_LABEL = "pos"
2 NEG_LABEL = "neg"
3 TARGET_COLUMN = "class"
4
5
6 def load_data(path: str) -> pd.DataFrame:
7     return pd.read_csv(path, na_values="na")
8
9
10 def split_xy(data: pd.DataFrame) -> tuple[pd.DataFrame, np.ndarray]:
11     x = data.drop(columns=[TARGET_COLUMN])
12     y = (data[TARGET_COLUMN] == POS_LABEL).astype(int).to_numpy()
13     return x, y
```

(二) 模型构建与代价函数

模型构建函数返回一个固定参数的 HistGradientBoostingClassifier。维修代价函数从混淆矩阵中取出 FP 和 FN, 并按照题目公式计算 Cost。

模型构建与维修代价计算

```
1 def make_model(random_state: int = 42) -> HistGradientBoostingClassifier:
2     return HistGradientBoostingClassifier(
3         max_iter=500,
4         learning_rate=0.04,
5         max_leaf_nodes=31,
6         l2_regularization=0.03,
7         random_state=random_state,
8     )
9
10
11 def aps_cost(y_true: np.ndarray, y_pred: np.ndarray) -> int:
12     tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
```

```
13     return int(10 * fp + 500 * fn)
```

(三) 阈值搜索与指标评估

阈值搜索函数遍历候选阈值, 并选择验证集 APS Cost 最低的阈值。评估函数统一计算 Cost、Precision、Recall、F1、Micro-F1、Macro-F1 和混淆矩阵, 便于训练和测试脚本复用。

阈值搜索

```
1 def threshold_grid() -> np.ndarray:
2     return np.r_[np.linspace(0.001, 0.1, 100), np.linspace(0.11, 0.9, 80)]
3
4
5 def select_threshold(
6     y_true: np.ndarray, scores: np.ndarray, thresholds: Iterable[float] |
7     None = None
8 ) -> tuple[float, int]:
9     best_threshold = 0.5
10    best_cost: int | None = None
11    for threshold in threshold_grid() if thresholds is None else thresholds:
12        pred = (scores >= threshold).astype(int)
13        cost = aps_cost(y_true, pred)
14        if best_cost is None or cost < best_cost:
15            best_cost = cost
16            best_threshold = float(threshold)
17    return best_threshold, int(best_cost)
```

(四) 训练与测试入口

main/train.py 先使用分层验证集选择阈值, 再用全量训练数据训练最终模型。模型包中保存最终模型、分类阈值、特征列顺序、验证集指标和随机种子。main/test.py 加载该模型包, 对齐特征列后输出预测概率, 并根据训练阶段选出的阈值得到预测类别。

训练阶段核心逻辑

```
1 x, y = split_xy(data)
2 x_train, x_valid, y_train, y_valid = train_test_split(
3     x,
4     y,
5     test_size=args.valid_size,
6     stratify=y,
7     random_state=args.random_state,
8 )
9
10 validation_model = make_model(args.random_state)
11 validation_model.fit(x_train, y_train)
12 valid_scores = validation_model.predict_proba(x_valid)[: , 1]
13 threshold, validation_cost = select_threshold(y_valid, valid_scores)
14 valid_pred = (valid_scores >= threshold).astype(int)
```

```
15 valid_metrics = evaluate_predictions(y_valid, valid_pred)
```

五、 实验结果显示

(一) 验证集结果

在分层验证集上，模型选出的最佳分类阈值为 0.0110。验证集混淆矩阵和主要指标如表 2 所示。

指标	数值
混淆矩阵 [TN FP FN TP]	[11468, 332, 4, 196]
APS Cost	5320
Precision	0.3712
Recall	0.9800
F1	0.5385
Micro-F1	0.9720
Macro-F1	0.7620

表 2: 分层验证集实验结果

从验证集结果可以看出，模型仅漏判 4 个正类样本，Recall 达到 0.9800。虽然由于正类比例很低，Precision 仍然低于 Recall，但该结果符合本任务对漏报更敏感的代价设置。

(二) 与 baseline 对比

在同一份 data/ml_train.csv 上评测时，本实验模型与 baseline 的结果如表 3 所示。

模型	阈值	混淆矩阵 [TN FP FN TP]	Cost	Precision	Recall	F1	Micro-F1	Macro-F1
Baseline	0.49	[57386, 1614, 44, 956]	38140	0.3720	0.9560	0.5356	0.9724	0.7607
本实验模型	0.0110	[57606, 1394, 6, 994]	16940	0.4162	0.9940	0.5868	0.9767	0.7874

表 3: 本实验模型与 baseline 的评测结果对比

对比结果表明，本实验模型将 APS Cost 从 38140 降低到 16940。漏报数 FN 从 44 降低到 6，误报数 FP 也从 1614 降低到 1394。同时，Precision、Recall、F1、Micro-F1 和 Macro-F1 均高于 baseline，说明模型并不是单纯通过增加误报换取召回率，而是在当前数据上同时改善了正类识别能力和整体分类效果。

六、 实验分析总结

本任务的核心挑战来自类别极不平衡和错误代价不对称。若只关注准确率或 Micro-F1，模型可能由于多数类 neg 占比过高而获得看似较好的整体指标，但这类模型容易漏判少数类 pos。在 APS 场景中，漏判相关故障的维修代价远高于误报，因此单纯使用默认阈值并不合适。

本实验使用梯度提升树处理表格特征和缺失值，并在验证集上直接以维修代价作为阈值选择标准。降低阈值后，模型会更积极地预测 pos，从而提高 Recall 并减少 FN。由于一次 FN 的代价为 500，而一次 FP 的代价仅为 10，减少漏报可以显著降低总 Cost。

从结果看, 阈值 0.0110 与本任务的代价函数较为匹配。模型在减少 FN 的同时也减少了 FP, 因此 Cost 的下降不是由误报增加换来的, 而是来自模型排序能力和阈值策略的共同改善。本文完成了 APS 空气压力系统故障检测实验报告的完整流程说明。实验使用梯度提升树分类器建立传统机器学习模型, 利用分层验证集搜索代价敏感分类阈值, 并以 APS Cost、Precision、Recall、F1、Micro-F1 和 Macro-F1 作为主要评估指标。结果显示, 本实验模型在验证集上取得 5320 的 Cost 和 0.9800 的 Recall, 在同一份训练数据评测中相较 baseline 将 Cost 从 38140 降低到 16940。

综上, 本实验方法能够较好适应 APS 故障检测中的不平衡二分类特点。它不依赖深度学习框架, 而是通过适合表格数据的梯度提升树和面向维修代价的阈值选择, 有效降低了漏报带来的高额成本。

NIJIA

参考文献

- [1] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [2] UCI Machine Learning Repository. Aps failure at scania trucks data set. <https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks>, 2017.

NIJU